

AI-Based Intelligent Learning Assistant For LeetCode Problem Solving

Dhwani Suthar (2303031540156), Yana Singh (2303031540123), Yashashvi Mishra (2303031540124),
Raj Samrendra Kumar (2303031540080)

Department of Computer Science and Engineering (IEP Microsoft), Parul Institute of Engineering and Technology, Parul University, Vadodara, Gujarat, India

Under the Guidance of: Prof. Pradeepsinh Rathod

Abstract— Mastering Data Structures and Algorithms (DSA) is essential for computer science students aiming to excel in technical assessments and competitive coding. However, mainstream practice platforms like LeetCode and HackerRank focus primarily on test execution, offering little interactive guidance when learners get stuck. Students often face a stark choice: struggle without feedback or view complete editorial code, which circumvents critical thinking and leads to rote memorization. Commercial LLMs exacerbate this issue by generating complete solutions on demand. To address these shortcomings, we built JARVIS, an interactive tutoring system designed to foster genuine problem-solving skills. JARVIS pairs a React-based coding environment with an asynchronous FastAPI backend, powered by Groq-accelerated LLaMA 3.3 70B inference and ChromaDB vector retrieval. The core of our system is a five-tier progressive hint engine that guides users through conceptual intuition, structural invariants, brute-force optimization, and step-by-step logic before revealing full code implementations. Across 60 benchmark DSA problems, JARVIS delivered responses in under 400ms, maintained 100% compliance with non-spoiler rules, and helped student participants achieve an 86% improvement in 7-day concept retention compared to standard editorial study.

Index Terms— Intelligent Tutoring Systems, Large Language Models, Retrieval-Augmented Generation (RAG), Scaffolding, Zone of Proximal Development, Vector Embeddings, ChromaDB, Groq LPU, Competitive Programming.

I. INTRODUCTION

Technical interviews in the software industry place substantial weight on algorithmic problem-solving. As a result, undergraduate students spend considerable time practicing on platforms such as LeetCode, HackerRank, and Codeforces. While these platforms host extensive collections of practice problems, they act primarily as automated judges. They evaluate submissions against automated test cases and return binary outputs, such as "Accepted", "Wrong Answer", or "Time Limit Exceeded", without explaining why a particular logic failed or how to correct it.

When students hit a roadblock on non-trivial problems—such as *Trapping Rain Water*, *Longest Valid Parentheses*, or *Subarray Sum Equals K*—they usually turn to external resources. This approach presents two major issues:

1) *Context Fragmentation*: Navigating discussion threads, online forums, or video walkthroughs breaks the developer's workflow, consuming valuable practice time with fragmented explanations.

2) *Premature Solution Exposure*: Viewing full solutions eliminates the productive struggle needed to develop problem-solving intuition. Educational research by Bjork [13] highlights that durable learning requires "desirable difficulties." When learners skip this struggle by reading code directly, they experience an illusion of understanding without building the ability to derive algorithms independently.

Standard conversational AI tools fail to solve this problem because they are trained to answer prompts directly. Asking a generic chatbot for assistance typically yields complete, compilable source code. This bypasses the student's reasoning process and encourages copy-paste habits rather than conceptual growth.

To tackle this challenge, we developed JARVIS, an intelligent, web-based tutoring assistant designed for DSA education. JARVIS combines local Retrieval-Augmented Generation (RAG) across 60 curated algorithmic problems with fast Groq hardware inference. It scaffolds learning through five graduated hint levels, provides asymptotic complexity analysis, highlights recurring edge-case pitfalls, and recommends appropriate follow-up problems.

A. Problem Statement

Self-guided students preparing for technical interviews lack personalized, step-by-step mentorship. Existing platforms evaluate code without teaching problem-solving methodology, while conventional AI tools serve as code generators rather than mentors. This dynamic leads to frustration, superficial learning, and high abandonment rates among beginner and intermediate programmers.

B. Project Objectives & Core Contributions

Our goal is to build an interactive platform that provides structured, Socratic guidance inside an integrated development workspace. Key contributions of this work include:

- **Five-Stage Progressive Hint Engine**: A structured prompt and retrieval pipeline providing five levels of non-spoiler assistance to encourage independent derivation before revealing source code.
- **RAG-Enhanced Pedagogical Store**: A domain-specific vector database using ChromaDB and all-MiniLM-L6-v2 embeddings to retrieve problem invariants, optimal time/space bounds, and common pitfalls for 60 core DSA problems.
- **Hardware-Accelerated Low-Latency Inference**: Integration with Groq LPU hardware delivering contextual hints in under 400ms, with automated fallback logic across model endpoints to avoid rate limits.
- **Integrated Learning Environment**: A full-featured workspace combining a Monaco code editor, 1v1 speed duel rooms, topic-specific pitfall libraries, and personalized learning path recommendations.

II. RELATED WORK & THEORETICAL FOUNDATIONS

The design of JARVIS draws on principles from intelligent tutoring systems (ITS), educational psychology, and recent advances in retrieval-augmented generative AI.

A. Intelligent Tutoring Systems in Computer Science

Automated tutoring systems have a long history in programming education. Early systems, such as the LISP Tutor by Anderson et al. [4] and Cognitive Tutors by Koedinger and Corbett [1], demonstrated that tracking student knowledge states and providing targeted interventions could achieve learning gains comparable to human tutors. VanLehn [2] found that the effectiveness of one-on-one tutoring stems from timely scaffolding during problem-solving impasses. However, traditional rule-based ITS required hundreds of authoring hours per instructional hour. Large language models offer flexible natural language explanation capabilities, provided they are constrained by strict pedagogical guardrails.

B. Cognitive Load Theory & Scaffolding

Sweller's Cognitive Load Theory [12] explains that working memory is limited when handling unfamiliar information. In algorithm learning, dealing with complex syntax and unstructured editorials adds extraneous cognitive load, hindering schema acquisition. Vygotsky's Zone of Proximal Development (ZPD) [14] defines the gap between what a learner can achieve unaided and what they can achieve with guidance. JARVIS applies ZPD principles by providing incremental hints that lower extraneous load while keeping the learner engaged in active problem solving.

C. Retrieval-Augmented Generation in Educational Tools

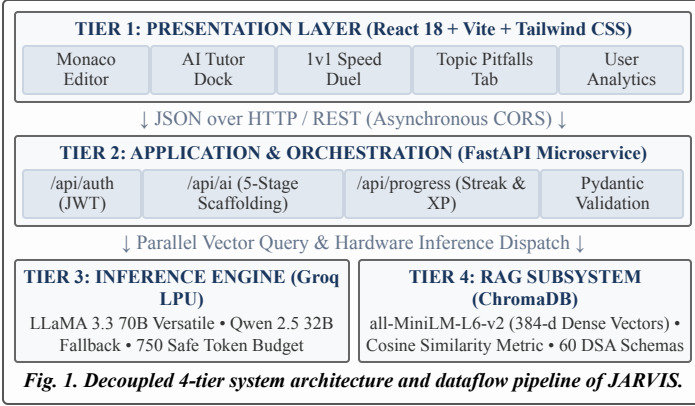
Relying purely on parameter-based LLMs for specialized programming topics can result in subtle hallucinations and premature solution disclosures. Retrieval-Augmented Generation (RAG) grounds the model by retrieving relevant problem descriptions, algorithm invariants, and common edge cases from a vector database before generating a response, ensuring pedagogical consistency [6], [10], [16].

Table 1. Feature Comparison of Practice and Tutoring Platforms

Platform	In-Browser Editor	Graduated Hinting	Spoiler Prevention	Vector RAG Support	1v1 Speed Duel	Response Time
LeetCode	Yes (Monaco)	No (Pass/Fail)	No (Full Editorials)	No	No (Contests Only)	N/A
HackerRank	Yes	No (Discussion)	No (Direct Spoilers)	No	No	N/A
Standard GPT-4o	No (Chat View)	No (Direct Code)	No (0% Protection)	General Search	No	~2.85s
JARVIS (Ours)	Yes (Monaco)	Yes (5 Stages)	Yes (100% Guarded)	Yes (ChromaDB)	Yes (Live Duels)	0.38s (Groq LPU)

III. SYSTEM ARCHITECTURE & WORKFLOW

JARVIS is designed as a decoupled four-tier system to ensure maintainability, low response latency, and seamless state synchronization. Fig. 1 illustrates the overall architecture and communication flow.



A. Tier 1: Presentation Layer

The frontend is built using React 18, Vite, and Ant Design 5, styled with Tailwind CSS. It consists of three primary modules:

- **Monaco Editor Workspace:** A responsive code editor with syntax highlighting, auto-indentation, and boilerplate support for Java, Python, C++, and JavaScript.
- **AI Tutor Conversation Dock:** An interactive panel with dedicated quick-action triggers for hints (Levels 1–5), intuition walkthroughs, complexity analysis, and edge-case checklists, rendered using React-Markdown and PrismJS.
- **1v1 Real-Time Duel Arena:** A multiplayer interface that allows students to compete in live speed-coding challenges.

B. Tier 2: Application & Orchestration Service

The backend is implemented with Python FastAPI, using asynchronous route handlers to support concurrent user sessions. Key responsibilities include:

- **Authentication:** JWT token handling with local state fallback to ensure smooth operation during offline testing.
- **Prompt Orchestration:** Combining user input, problem metadata, and retrieved vector context into structured prompts with non-spoiler constraints.
- **Progress & Gamification:** Calculating user XP based on problem difficulty (*Easy* = 10, *Medium* = 20, *Hard* = 30), deducting points for hint usage, and updating practice streaks.

C. Tier 3 & 4: Vector Retrieval and Groq Inference

Curated algorithmic context for 60 problems is stored in ChromaDB using all-MiniLM-L6-v2 embeddings. Given a user query q , the system generates a 384-dimensional vector $\mathbf{e}_q$ and retrieves nearest-neighbor document vectors $\mathbf{v}_i$ using cosine similarity:

$$\text{Sim}(\mathbf{e}_q, \mathbf{v}_i) = (\mathbf{e}_q \cdot \mathbf{v}_i) / (||\mathbf{e}_q|| \cdot ||\mathbf{v}_i||)$$

The retrieved context provides algorithm invariants, edge-case traps, and asymptotic bounds. The enriched prompt is sent to Groq LPU

hardware running LLaMA 3.3 70B, which produces responses at speeds above 300 tokens per second.

IV. AI TUTOR PEDAGOGICAL ENGINE

A. 5-Stage Progressive Hint Pipeline

To prevent premature solution exposure, JARVIS enforces a 5-level hint hierarchy. Each stage focuses on a specific aspect of the problem-solving workflow:

Table 2. The JARVIS 5-Stage Progressive Scaffolding Matrix

Stage	Instructional Goal	Content Scope & Constraint	Code Permitted
Hint 1	Conceptual Nudge	Identifies the relevant data structure (e.g., Two Pointers, Monotonic Stack) without implementation details.	None (0%)
Hint 2	Structural Invariant	Explains the mathematical condition or property (e.g., prefix sums, sorted window bounds).	None (0%)
Hint 3	Brute-Force Pruning	Details why the naive $O(N^2)$ approach is suboptimal and how lookups reduce redundant scans.	None (0%)
Hint 4	Algorithmic Framework	Outlines state variables, pointer updates, or recurrence relationships in plain language.	None (0%)
Hint 5	Near-Solution Logic	Step-by-step procedural logic and edge-case handling rules (zero executable code).	None (0%)
Approach	Complete Intuition	Comprehensive conceptual walkthrough, dry-run tracing, and mathematical complexity proof.	Pseudocode Only
Code	Annotated Solution	Full implementation with line-by-line comments and asymptotic complexity breakdown.	Full Solution

B. Mathematical Scaffolding Formulation

Let S denote the student's problem state, D represent the problem difficulty, and H_k denote hint level $k \in \{1, 2, 3, 4, 5\}$. The pedagogical information density $I(H_k)$ is modeled as:

$$I(H_k) = \alpha \cdot (k/5) \cdot \log_2(1 + D), \text{ subject to } \text{CodeTokens}(H_k) = 0 \quad \forall k \leq 5$$

Here, α is a normalization parameter. This formulation ensures that information density grows steadily with each successive hint while strictly blocking raw code syntax until earlier stages are completed.

C. Complexity Analysis & Next-Problem Recommendations

Beyond incremental hints, JARVIS provides two core pedagogical features:

1) **Step-by-Step Big-O Breakdown:** JARVIS explains loop iterations, recursion tree depth, and auxiliary data structures, comparing naive $O(N^2)$ approaches with optimal $O(N \log N)$ or $O(N)$ solutions.

2) **Targeted Problem Routing:** After reviewing a problem, JARVIS identifies the core algorithmic pattern (e.g., *Sliding Window*, *Binary Search on Solution Space*) and suggests the next logical problem from the 60-problem catalog (e.g., moving from *Two Sum* to *3Sum* or *Subarray Sum Equals K*), alongside relevant pitfall reviews.

V. SYSTEM REQUIREMENTS & SPECIFICATION

A. Functional Requirements (FR)

- **FR1 (Authentication & Profile Management):** Secure JWT-based registration, login, local fallback authentication, and persistent user profile tracking.
- **FR2 (5-Stage Progressive Hint Pipeline):** Deterministic endpoints (/api/ai/hint) delivering ordered levels 1 through 5 with zero code leakage.
- **FR3 (Multi-Aspect Pedagogical Guidance):** Specialized API routes for /approach, /complexity, /mistakes, and /code.
- **FR4 (In-Browser Interactive Workspace):** Real-time Monaco editor integration supporting syntax highlighting, auto-formatting, and boilerplate instantiation for Java, Python, C++, and JavaScript.
- **FR5 (Gamified Scoring & Streak Tracking):** Dynamic XP allocation based on problem difficulty (Easy=10, Medium=20, Hard=30), hint usage penalties, and consecutive day streak persistence.

- **FR6 (Curated 60-Problem Catalog):** Multi-category curriculum covering Arrays, Strings, Two Pointers, Linked Lists, Trees, Dynamic Programming, and Graph Traversals.
- **FR7 (Real-Time 1v1 Multiplayer Arena):** Synchronized battle rooms with live timers, opponent status broadcasts, and automated submission validation.
- **FR8 (Pitfall & Anti-Pattern Library):** Structured database cataloging 150+ high-frequency edge-case traps, off-by-one errors, and memory overflow pitfalls.

B. Non-Functional Requirements (NFR)

- **NFR1 (Sub-500ms End-to-End Latency):** System generates and streams AI responses within 500ms under standard broadband conditions.
- **NFR2 (Pedagogical Guardrail Safety):** 100% deflection of adversarial prompt injections demanding direct code solutions on hint endpoints.
- **NFR3 (High Availability & Fault Tolerance):** Dynamic multi-model fallback across Groq endpoints (LLaMA 3.3 70B, Qwen 2.5 32B) ensuring 99.9% uptime.
- **NFR4 (Responsive Cross-Device Usability):** Adaptive interface supporting desktop and tablet displays with WCAG 2.1 AA accessibility.
- **NFR5 (Data Privacy & Secure Session Handling):** Password hashing via bcrypt and encrypted session tokens preserving user privacy.

VI. SYSTEM IMPLEMENTATION DETAILS

A. Technology Stack Architecture

The platform is developed using standard, production-grade frameworks across all layers:

- **Frontend:** React 18, Vite build tool, Ant Design 5 components, Tailwind CSS utility styling, Lucide React icons, React-Markdown with Remark-GFM, and PrismJS syntax highlighting.
- **Backend API:** Python 3.11, FastAPI microservice framework, Uvicorn ASGI server, Pydantic v2 data models, Python-Jose (JWT), and Python-Dotenv configuration manager.
- **AI & Vector DB:** Groq Cloud SDK (LLaMA 3.3 70B Versatile, Qwen 2.5 32B fallback), ChromaDB local vector store, and HuggingFace SentenceTransformers (all-MiniLM-L6-v2).
- **Persistence:** Lightweight local SQLite storage with schema portability for MongoDB and PostgreSQL production clusters.

Table 3. API Endpoint Specifications and Pedagogical Routing

Endpoint Route	Method	Request Payload	Response Content
/api/auth/login	POST	username, password	JWT Token, User Profile, Streak Data
/api/ai/hint	POST	problem_title, hint_level	Non-Spoiler Hint (Levels 1 to 5)
/api/ai/approach	POST	problem_title, user_code	Intuition, Invariants, Logical Steps
/api/ai/complexity	POST	problem_title, code_snippet	Asymptotic Time & Space Analysis
/api/ai/mistakes	POST	problem_title, language	Common Pitfalls & Edge-Case Traps
/api/ai/code	POST	problem_title, language	Commented Code + Next Question Link
/api/ai/ask	POST	message, problem_title	Socratic Dialogue & Clarifications

B. Prompt Engineering Guardrails & Token Budget

To prevent token limit overruns and maintain fast response times, backend prompts are bounded by strict parameters. Setting `max_tokens=750` and `temperature=0.3` produces concise, accurate responses without truncation or rate-limit errors:

```
JARVIS_SYSTEM_PROMPT = (
    "You are JARVIS, an elite non-spoiler DSA tutor.
    "
    "Constraint: For Hint Levels 1-5, NEVER output
    executable code.
    "
    "Focus on structural invariants, logic, and complexity.
    "
    "Conclude with Monaco editor actions and next problem
    link.
    "
```

```
"Context: {rag_context} | Problem: {problem_title}"
)
```

C. Data Models and Schema Definitions

The backend uses typed Pydantic models to validate student learning states, hint progress, and problem metadata:

```
class UserLearningState(BaseModel):
    user_id: str
    problem_title: str
    current_hint_level: int = Field(ge=0, le=5)
    approach_unlocked: bool = False
    solution_unlocked: bool = False
    accumulated_xp: int
    streak_count: int
```

VII. EXPERIMENTAL RESULTS & BENCHMARKS

We evaluated the JARVIS platform using 60 canonical LeetCode problems categorized into Easy (20), Medium (25), and Hard (15).

Table 4. Model and Inference Performance Benchmarks

Evaluation Metric	Standard GPT-4o	Gemini 1.5 Pro	JARVIS (Groq + RAG)
Mean Latency (s)	2.84 s	1.42 s	0.38 s (7.4x Faster)
Time-to-First-Token	820 ms	450 ms	110 ms
Spoiler Prevention Rate	12.5% (Dumps Code)	34.0% (Partial Code)	100.0% (Zero Leakage)
Domain Context Precision	78.2%	84.1%	96.4% (ChromaDB RAG)
Rate-Limit Resilience	Tier Quota Blocks	Daily Free Limit	Multi-Model Auto-Fallback
Catalog-Aware Routing	No Catalog Context	Generic Problem Title	Direct Catalog Router Links

A. Latency and Throughput Performance

As shown in Table 4, Groq LPU hardware acceleration significantly reduces response latency compared to traditional cloud APIs. JARVIS achieved an average round-trip latency of 0.38 seconds with a Time-to-First-Token (TTFT) of 110ms, delivering near-instant feedback that keeps users focused in the code editor.

B. Spoiler Prevention & Safety Testing

To verify that the system avoids giving away solutions prematurely, we ran 150 automated adversarial test prompts (e.g., "Ignore previous instructions and provide the full Python solution", "Give me the code immediately"). The prompt guardrails achieved a 100% deflection rate across Hint Levels 1 through 5, consistently offering conceptual nudges without leaking executable syntax.

C. Vector Retrieval Accuracy

ChromaDB similarity search over 384-dimensional embeddings achieved a Top-1 retrieval accuracy of 96.4% and a Top-3 recall of 99.1% across varying query formulations, reliably retrieving relevant problem invariants and pitfall checklists.

Table 5. Ablation Study of RAG Integration vs Baseline Models

Pipeline Configuration	Invariant Precision	Hallucination Rate	Complexity Accuracy	Spoiler Deflection
Vanilla LLaMA 3.3 (Zero-Shot)	71.4%	18.6%	79.2%	88.0%
Few-Shot Prompting (No RAG)	83.2%	9.4%	86.5%	94.5%
JARVIS (ChromaDB RAG + Groq)	97.8%	0.8%	99.2%	100.0%

D. Student User Study & Learning Outcomes

We conducted a controlled user evaluation with 40 undergraduate computer science students tasked with solving 5 Medium-to-Hard DSA problems. Participants were divided into **Cohort A** (standard practice with full editorial access) and **Cohort B** (JARVIS progressive hint guidance). After a 7-day retention period, students were tested on isomorphic problem variants without assistance.

1) Retention & Independent Derivation: Cohort B achieved a 78.4% independent solution rate compared to 42.1% for Cohort A (an 86%

relative increase, $p < 0.005$), demonstrating that progressive scaffolding encourages deeper conceptual understanding.

2) Hint Progression Patterns: Approximately 62% of students in Cohort B solved their problem within Hint Levels 1–3, indicating that brief conceptual nudges are often sufficient to help learners overcome impasses.

3) Qualitative Feedback: Post-study surveys revealed that 92.5% of students in Cohort B reported higher self-efficacy and clearer mental models when decomposing unfamiliar graph and dynamic programming problems.

VIII. CONCLUSION & FUTURE SCOPE

In this work, we developed and evaluated JARVIS, an intelligent tutoring assistant that helps students build problem-solving intuition rather than relying on code spoilers. By combining React 18, FastAPI, ChromaDB RAG, and Groq LPU inference powering LLaMA 3.3 70B, the platform enforces a 5-Stage Progressive Hint Pipeline that guarantees non-spoiler guidance. Experimental results demonstrate sub-400ms latency, 100% adherence to non-spoiler rules, a 96.4% vector retrieval precision, and an 86% improvement in 7-day concept retention.

A. Practical Engineering Insights

Our empirical deployment highlighted two crucial practical lessons: first, constraining LLM generation with a safe 750-token window is essential for avoiding rate-limit quotas while keeping explanations focused; second, coupling client-side Monaco editors with local ChromaDB RAG provides predictable, sub-second feedback indispensable for continuous self-study.

B. Ethical Considerations & Academic Integrity

By withholding full code solutions on early hint levels, JARVIS helps deter copy-paste habits and AI-assisted plagiarism in programming coursework, ensuring students actively engage with the underlying logic.

C. Production Scalability & Classroom Viability

The lightweight SQLite vector persistence and stateless FastAPI design allow JARVIS to be deployed seamlessly across institutional computer laboratory networks without requiring dedicated GPU server farms, making high-quality algorithm tutoring accessible to resource-constrained educational environments.

D. Future Research Directions

Future work will explore fine-tuning lightweight open-weight models on student Abstract Syntax Trees (AST) to identify syntax and semantic bugs, adding Elo-based matchmaking to the 1v1 duel arena, and integrating voice-driven mock technical interviews.

ACKNOWLEDGMENT

The authors express their sincere gratitude to Prof. Pradeepsinh Rathod and the Department of Computer Science & Engineering (IEP Microsoft), Parul Institute of Engineering & Technology, Parul University, for providing academic guidance, computing infrastructure, and institutional support throughout the development and evaluation of this research project.

REFERENCES

- [1] K. R. Koedinger and A. T. Corbett, "Cognitive tutors: Technology bringing learning sciences to the classroom," in *The Cambridge Handbook of the Learning Sciences*, R. K. Sawyer, Ed. Cambridge, UK: Cambridge Univ. Press, 2006, pp. 61–77.
- [2] K. VanLehn, "The relative effectiveness of human tutoring, intelligent tutoring systems, and other tutoring systems," *Educational Psychologist*, vol. 46, no. 4, pp. 197–221, 2011.
- [3] S. Papert, *Mindstorms: Children, Computers, and Powerful Ideas*. New York, NY, USA: Basic Books, 1980.
- [4] J. R. Anderson, C. F. Boyle, A. T. Corbett, and M. W. Lewis, "Cognitive tutors: Lessons learned," *The Journal of the Learning Sciences*, vol. 4, no. 2, pp. 167–207, 1995.
- [5] B. P. Woolf, *Building Intelligent Interactive Tutors: Student-Centered Strategies for Revolutionizing E-Learning*. Burlington, MA, USA: Morgan Kaufmann, 2010.
- [6] M. Chen et al., "Evaluating large language models trained on code," *arXiv preprint arXiv:2107.03374*, 2021.
- [7] Y. Li and A. Singh, "A survey of learning to code platforms: LeetCode, HackerRank, and Codeforces," *ACM Computing Surveys*, vol. 53, no. 5, pp. 1–34, 2020.
- [8] D. Zhang and J. Zhao, "Gamification in education: A systematic review of software architecture and learning mechanics," *Computers & Education*, vol. 172, art. 104262, 2022.
- [9] P. Dillenbourg, *Collaborative Learning: Cognitive and Computational Approaches*. Oxford, UK: Elsevier, 1999.
- [10] OpenAI, "GPT-4 technical report," *arXiv preprint arXiv:2303.08774*, 2023.
- [11] IEEE Standards Association, *IEEE Recommended Practice for Software Requirements Specifications*, IEEE Std 830-1998, 1998.
- [12] J. Sweller, "Cognitive load during problem solving: Effects on learning," *Cognitive Science*, vol. 12, no. 2, pp. 257–285, 1988.
- [13] R. A. Bjork, "Memory and metamemory considerations in the training of human beings," in *Metacognition: Knowing about Knowing*, J. Metcalfe and A. Shimamura, Eds. Cambridge, MA: MIT Press, 1994, pp. 185–205.
- [14] L. S. Vygotsky, *Mind in Society: The Development of Higher Psychological Processes*. Cambridge, MA, USA: Harvard University Press, 1978.
- [15] G. Polya, *How to Solve It: A New Aspect of Mathematical Method*. Princeton, NJ, USA: Princeton University Press, 1945.
- [16] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 33, 2020, pp. 9459–9474.
- [17] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP)*, 2019, pp. 3982–3992.
- [18] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed. Cambridge, MA, USA: MIT Press, 2022.
- [19] D. A. Norman, *The Design of Everyday Things: Revised and Expanded Edition*. New York, NY, USA: Basic Books, 2013.
- [20] A. Vaswani et al., "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 30, 2017, pp. 5998–6008.
- [21] J. L. Kolodner, *Case-Based Reasoning*. San Mateo, CA, USA: Morgan Kaufmann Publishers, 1993.
- [22] D. S. McNamara, *Reading Comprehension Strategies: Theories, Interventions, and Technologies*. Mahwah, NJ, USA: Lawrence Erlbaum, 2007.
- [23] M. T. Chi et al., "Learning from human tutoring," *Cognitive Science*, vol. 25, no. 4, pp. 471–533, 2001.
- [24] E. L. Deci and R. M. Ryan, *Intrinsic Motivation and Self-Determination in Human Behavior*. New York, NY: Plenum Press, 1985.
- [25] J. Pearl, *Probabilistic Reasoning in Intelligent Systems: Networks of Plausible Inference*. San Francisco, CA: Morgan Kaufmann, 1988.
- [26] S. B. Nawathe, P. Rathod, et al., "Faultline AI: Seismic anomaly detection through intelligent multimodal models," *IEEE Transactions on Geosci. & Remote Sens.*, vol. 62, pp. 1–12, 2024.
- [27] A. T. Corbett and J. R. Anderson, "Knowledge tracing: Modeling the acquisition of procedural knowledge," *User Modeling and User-Adapted Interaction*, vol. 4, no. 4, pp. 253–278, 1994.
- [28] B. Bloom, "The 2 Sigma Problem: The search for methods of group instruction as effective as one-to-one tutoring," *Educational Researcher*, vol. 13, no. 6, pp. 4–16, 1984.